

MoDIFI User Guide

Multi-omics Differential Inference for Functional Interpretation

Cell Line Analysis Pipeline

Version: March 2026

github.com/tris-10/MoDIFI

[Arvinden et al., bioRxiv 2026](#)

1. What Is MoDIFI and Why Would I Use It?

MoDIFI (**M**ulti-omics **D**ifferential **I**nterference for **F**unctional **I**nterpretation) is a Bayesian statistical pipeline for prioritizing noncoding genomic regions and cell lines most likely to show cell-type-specific regulatory activity. It is designed to help researchers choose which in vitro cell line model to use (and which specific regulatory loci to focus on) when planning high-throughput functional assays such as massively parallel reporter assays (MPRAs) or other analysis and experimentation.

i BACKGROUND

Noncoding variants associated with neurodevelopmental disorders (NDDs) often exert regulatory effects in a cell-type-specific manner. This means a variant might strongly affect gene regulation in a neuronal cell line but show no effect in a non-neuronal line. Selecting the wrong cell line for a functional assay wastes time and resources. MoDIFI addresses this by integrating three orthogonal omics modalities — RNA-seq (gene expression), ATAC-seq (chromatin accessibility), and Hi-C (3D genome organization) — to compute a per-loop, per-cell-line regulatory activity score.

1.1 What data does MoDIFI need?

MoDIFI requires three matched omics datasets per cell line:

- **RNA-seq** — gene-level count files (RSEM *.genes.results or similar).
- **ATAC-seq** — aligned BAM files and called narrow peaks (*.narrowPeak.gz).
- **Hi-C** — chromatin loop calls in BEDPE format (*.bedpe.gz).
- **Resource files** (provided, see Section 3): Gnocchi prior file, promoter annotation file, hg38 gene annotation.

1.2 What does MoDIFI output?

For each specified target vs. reference cell line pair, MoDIFI produces a tab-separated output file containing:

- **MoDIFI PIP** — a Posterior Inclusion Probability (0–1) representing how strongly a Hi-C loop shows differential regulatory activity in the target versus reference cell line. Higher PIP = more likely to be functionally relevant in the target.
- **Per-loop scores** — each loop entry integrates the differential ATAC-seq accessibility and differential RNA-seq expression for nearby genes, weighted by prior constraint information (Gnocchi scores).

In most downstream analyses, MoDIFI_loop_(Target)_vs_(Reference).tsv contains the maximum MoDIFI score for each Hi-C loop, while MoDIFI_all_(Target)_vs_(Reference).tsv lists all MoDIFI scores for loops linked to multiple ATAC-seq peak combinations.

For example, MoDIFI_all_IMR90_vs_GM12878.tsv is a tab-delimited file reporting all Hi-C loop–ATAC peak combinations evaluated in the IMR90 versus GM12878 comparison; each row represents a chromatin loop paired with a specific ATAC-seq peak combination and includes genomic coordinate columns (chr, start, end), ATAC-seq-specific columns (X_ATAC_*,

Y_ATAC_*), RNA-seq-specific columns (RNAseq_*), Bayes factor columns (*_BF), the Gnocchi prior, and the final MoDIFI score*.

1.3 High-level workflow overview

Before diving into steps, here is the full sequence of what MoDIFI does from start to finish:

1. **ATAC-seq preprocessing:** Peaks are extracted from each cell line, merged into a consensus peak set, and read counts per peak are quantified from BAM files.
2. **Differential analysis (DESeq2):** Differential accessibility (ATAC) and differential expression (RNA) are computed between each target/reference pair using DESeq2 (Love et al., 2014).
3. **Controlling inflation:** A mixture model is applied (van Iterson et al., 2017) is applied to adjust for bias and inflation in the differential statistics.
4. **Regulatory linking:** ATAC peaks near promoters are linked to Hi-C loops, connecting chromatin accessibility to 3D genome structure.
5. **Bayesian scoring (MoDIFI):** Our combined model (VR et al., 2026) provides per-loop Bayes factors differential omics signals. Empirical priors derived from Gnocchi (Chen et al., 2023) population-genetics constraint scores are applied to compute final Posterior Inclusion Probabilities (PIPs).
6. **Output:** MoDIFI TSV files are written to the output directory for each cell line pair.

i BACKGROUND

The Gnocchi score is a measure of regional constraint derived from population genetics — regions under strong negative selection have higher Gnocchi scores. MoDIFI converts these to percentile-ranked priors so that loops in more constrained regions receive higher prior weight. This is controlled by the `prior_file` parameter in `modifi.config`.

2. Prerequisites: What You Need Before Running

Before running MoDIFI, ensure you have the following installed and accessible on your system.

2.1 Required software

Software	Version	Notes
Nextflow	≥ 22.10	Workflow manager that runs the pipeline
Singularity	≥ 3.7	Container runtime (all tools run inside <code>modifi.sif</code>)

Software	Version	Notes
SLURM (or other HPC) or local	any	The example config uses SLURM; can adapt for other executors or run locally by editing the configuration file

i BACKGROUND

Singularity is a container system commonly used on shared HPC clusters. Unlike Docker, it does not require root privileges. All Python and R dependencies (including DESeq2, samtools, bedtools, bacon, etc.) are pre-installed inside the container image (modifi.sif). You do not need to install these yourself.

Nextflow is a workflow manager written in Groovy/Java that handles job submission, parallelization, and error recovery. It requires Java 11+ to be available on your system. Run: `nextflow -version` to check.

The pipeline has been tested with Nextflow 25.10.2. Users can install Nextflow following the official guide: <https://www.nextflow.io/docs/latest/install.html>. For containerized execution, users may install Singularity/Apptainer following the official documentation (<https://apptainer.org/docs/>).

In the provided example configuration, the pipeline is set to use the SLURM executor (`executor = 'slurm'`), which allows Nextflow to submit jobs to a SLURM scheduler automatically. However, MoDIFI does not require SLURM. Users who wish to run the pipeline without a scheduler such as on a workstation or for small test datasets can switch to local execution either by editing the configuration file (`executor = 'local'`) or by overriding the executor at runtime using `-process.executor local`. This flexibility allows MoDIFI to run in any environment supported by Nextflow, from HPC clusters to single-node systems.

2.2 Required resource files

The following files must be present in the `resources/` directory before running. These are not generated by the pipeline — you download or prepare them in advance.

File	Location	Source	Description
modifi.sif	Working directory root	Download (see below)	Singularity container image with all software
Gnocchi.tsv	resources/	Download (see below)	Population-genetics constraint prior
Promoter/Promoter.tsv	resources/Promoter/	Download (see below)	Protein-coding gene promoter annotations (hg38)
hg38_annotation.txt	resources/	Download (see below)	hg38 gene annotation for RNA-seq DESeq2

Download resource files:

modifi.sif: <https://zenodo.org/records/19926880/files/modifi.sif?download=1>

Gnocchi.tsv: <https://zenodo.org/records/19926880/files/Gnocchi.tsv?download=1>

Promoter.tsv: <https://github.com/tris-10/MoDIFI/blob/main/resources/Promoter/Promoter.tsv>

hg38_annotation.txt: https://github.com/tris-10/MoDIFI/blob/main/resources/hg38_annotation.txt

Container image download:

The pipeline uses a prebuilt Singularity/Apptainer container image (modifi.sif). As this is a large file, users are recommended to download it explicitly using `wget` or `curl` rather than through a web browser. For example:

```
wget <CONTAINER_URL> -O modifi.sif
```

3. Quickstart: Running the Provided Test Example

We strongly recommend running MoDIFI on the provided test dataset before applying it to your own data. This confirms your environment is set up correctly and gives you a reference for what the output should look like.

3.1 Example Data

Publicly available ENCODE datasets are provided as a minimal test dataset to allow users to verify correct installation and execution of the MoDIFI pipeline. Two ENCODE cell lines (IMR90 and GM12878) are used, consistent with those referenced in `modifi_example.config`, and include RNA-seq, ATAC-seq, and Hi-C data.

Note: The directory paths shown below are examples only. Users may download the data to any location, provided that the full paths are correctly specified in `modifi_example.config`.

Example download command:

```
wget https://www.encodeproject.org/files/ENCFF019KLP/@@download/ENCFF019KLP.tsv \
-O ./resources/imr90/rna_seq/ENCFF019KLP.tsv
```

RNA-seq

Cell Line	Source URL	Destination Directory
IMR90 Replicate 1	https://www.encodeproject.org/files/ENCFF019KLP/@@download/ENCFF019KLP.tsv	./resources/imr90/rna_seq/
IMR90 Replicate 2	https://www.encodeproject.org/files/ENCFF268IJX/@@download/ENCFF268IJX.tsv	./resources/imr90/rna_seq/
GM12878 Replicate 1	https://www.encodeproject.org/files/ENCFF014JRQ/@@download/ENCFF014JRQ.tsv	./resources/gm12878/rna_seq/

ATAC-seq

Cell Line	Source URL	Destination Directory
IMR90 Replicate 1 (BAM)	https://www.encodeproject.org/files/ENCFF848XMR/@@download/ENCFF848XMR.bam	./resources/imr90/atac_seq/bam/
IMR90 Replicate 2 (BAM)	https://www.encodeproject.org/files/ENCFF715NAV/@@download/ENCFF715NAV.bam	./resources/imr90/atac_seq/bam/
IMR90 Peak Calls (BED.gz)	https://www.encodeproject.org/files/ENCFF830JMV/@@download/ENCFF830JMV.bed.gz	./resources/imr90/atac_seq/
GM12878 Replicate 1 (BAM)	https://www.encodeproject.org/files/ENCFF646NWY/@@download/ENCFF646NWY.bam	./resources/gm12878/atac_seq/bam/
GM12878 Replicate 2 (BAM)	https://www.encodeproject.org/files/ENCFF415FEC/@@download/ENCFF415FEC.bam	./resources/gm12878/atac_seq/bam/
GM12878 Peak Calls (BED.gz)	https://www.encodeproject.org/files/ENCFF478ZAW/@@download/ENCFF478ZAW.bed.gz	./resources/gm12878/atac_seq/

Hi-C

Cell Line	Source URL	Destination Directory
IMR90	https://www.encodeproject.org/files/ENCFF267VBT/@@download/ENCFF267VBT.bedpe.gz	./resources/imr90/hic/
GM12878	https://www.encodeproject.org/files/ENCFF661SAZ/@@download/ENCFF661SAZ.bedpe.gz	./resources/gm12878/hic/

Full file paths in modifi_example.config

All input files specified in `modifi_example.config` must be provided as **full file paths**. The example paths shown below correspond to **ATAC-seq BAM files**. The same path structure and specification apply to other input data types, including **ATAC-seq peak files**, **RNA-seq count files**, and **Hi-C loop files**, as defined in the configuration fields `ATACBEDFile`, `RNACountFile`, and `HiCLoopsFile`.

```
ATACBAMFiles = [
    "IMR90":    "${resources_dir}/resources/imr90/atac_seq/bam/*.bam",
    "GM12878":  "${resources_dir}/resources/gm12878/atac_seq/bam/*.bam"
]
```

Once the test data is in place, the steps to run it are:

Step 1 — Clone the repository:

```
git clone https://github.com/tris-10/MoDIFI.git
cd MoDIFI
```

Step 2 — Download the container and resource files

Place `modifi.sif` in the `MoDIFI/` root directory. Place resource files in `resources/` as described in Section 2.2.

Step 3 — Organize the test input data

Place the ENCODE test data files following the directory layout in Section 4. Update `modifi_example.config` to point to the test data paths.

Step 4 — Configure Singularity bind mount

Edit the `singularity { runOptions }` parameter in your config file to bind-mount the parent directory of your data. See Section 5.4 for details.

Step 5 — Run the pipeline:

```
nextflow run modifi.nf -c modifi_example.config
```

Step 6 — Verify output

Output files will appear in the directory specified by `output_dir` in the config. Compare against the expected example output (see analyst note above).

4. Directory Layout

The MoDIFI pipeline expects the following directory structure. You do not need to create all of these — the pipeline generates intermediate files automatically — but you need to set up the input data locations before the first run.

```
MoDIFI/
├── modifi.nf                # Main Nextflow pipeline
├── modifi.config            # Configuration file (you edit this)
├── modifi.sif              # Singularity container image (you download this)
├── scripts/               # Pipeline scripts (do not modify)
│   ├── *.py
│   └── *.R
├── resources/             # Resource and reference files
│   ├── SampleInfo.tsv     # AUTO-GENERATED by pipeline (do not create manually)
│   ├── SamplePair.tsv     # You create this or it is auto-generated (see Section 6)
│   ├── hg38_annotation.txt # Download — gene annotation for RNA-seq
│   ├── Gnocchi.tsv        # Download — constraint prior
│   └── Promoter/
│       └── Promoter.tsv    # Download — promoter annotations
└── output/               # All pipeline outputs go here (created automatically)
```

Files marked 'AUTO-GENERATED' are created by the pipeline during execution. Do not create or edit them before running — the pipeline will overwrite them.

SamplePair.tsv is special: you can create it in advance to specify which cell line pairs to compute (see Section 6.1), or let the pipeline generate all pairwise combinations automatically.

5. Organizing Your Input Data

Each cell line requires three types of input files. You specify the paths to these files in `modifi_example.config` — the pipeline does not require any specific directory structure for your raw data, as long as the paths in the config are correct.

5.1 Required input file types

Omics type	File format	Config parameter
RNA-seq	*.genes.results (RSEM) or *.tsv	RNACountFile
ATAC-seq BAMs	*.bam (aligned)	ATACBAMFiles
ATAC-seq peaks	*.narrowPeak.gz (called peaks)	ATACBEDFile
Hi-C loops	*.bedpe.gz (loop calls)	HiCLoopsFile

5.2 Recommended directory layout for input data

While a standardized directory structure is not strictly required, we recommend organizing input files in a consistent and logical layout to improve readability and maintainability. The example configuration below illustrates one possible organization.

Note: In the actual configuration used in this work, all omics input files are specified using absolute (full) paths, and the pipeline does not rely on this directory structure programmatically. The layout described here is therefore recommended for user convenience only, not a requirement.

Example directory structure

```
resources/
├── gml2878/
│   ├── rna_seq/ (contains *.genes.results or *.tsv files)
│   ├── atac_seq/
│   │   ├── bam/ (contains *.bam files)
│   │   └── *.narrowPeak.gz
│   └── hic/
│       └── *.bedpe.gz
└── imr90/
    └── ... (same structure)
```


6. Configuring the Pipeline (modifi.config)

The pipeline behavior is controlled by the file `modifi.config` (an example is provided as `modifi_example.config` in the repository). You will copy and modify this file for your own run. Below is a walkthrough of every parameter you need to set.

6.1 Parameters section

These are the settings you are most likely to change:

```
params {

    // — Paths —————
    resources_dir = "${projectDir}/resources" // Where resource files live
    output_dir    = "${projectDir}/output/MyRun" // Where results will be written
    script_dir    = "${projectDir}/scripts" // Do not change

    SampleInfo = "${resources_dir}/SampleInfo.tsv" // Auto-generated; do not change
    SamplePair = "${resources_dir}/SamplePair.tsv" // See Section 6.3
    prior_file = "${resources_dir}/Gnocchi.tsv" // Downloaded prior file

    // — Cell lines and replicates —————
    // List each cell line name and its number of ATAC-seq replicates:
    RepNUM_atac = ['GM12878': 2, 'IMR90': 2]

    // List each cell line and its number of RNA-seq replicates:
    RepNUM_rna = ['GM12878': 1, 'IMR90': 2]

    // — ATAC-seq settings —————
    ATACpeakFile = '/atac_seq/*.bed.gz' // Glob pattern appended to
resources_dir/[cell]/
    ATACSeq      = 'ATACseq' // Label used in DESeq2 output filenames
    atac_minQ    = 5 // Minimum peak quality score (Q < 5
filtered out)
    ATAC_Key_col = 'Region' // Key column for DESeq2 output

    // — RNA-seq settings —————
    RNAFilePattern = '/*tsv' // Glob pattern for RNA count files
    RNASeq         = 'RNAseq' // Label used in DESeq2 output filenames
    RNA_Key_col    = 'GeneID' // Key column for DESeq2 output
    RNA_ANN_File   = "${resources_dir}/hg38_annotation.txt"
    RNA_quantification = 'expected_count' // Column name in RSEM output to use
}
```

6.2 Process resource settings

The process block controls how much compute resource each pipeline step gets. You will need to adjust these to match your HPC environment:

```
process {
    executor = "slurm" // Change to 'local', 'pbs', 'lsf', etc. as
needed
```

```
errorStrategy = 'finish'          // Continue other jobs if one fails
clusterOptions = "--mail-user your.email@institution.edu --mail-type FAIL"

withName: MapATACWithPRO          { cpus = 4; memory = 24.GB }
withName: MapATACWithHiCWithPRO { memory = 32.GB; maxForks = 4; cpus = 3 }
withName: AdjustZInflation        { memory = 32.GB; maxForks = 4; cpus = 3 }
withName: calMoDIFY               { memory = 32.GB; maxForks = 4; cpus = 3 }
withName: runDEseq                { memory = 32.GB; maxForks = 4; cpus = 3 }
}
```

The memory and CPU settings specified in `modifi.config` were used in the current analysis of kidney and neuron cell lines. Additional testing showed that the pipeline also ran successfully when memory allocation was reduced to 8 GB.

6.3 Singularity settings

The `singularity` block configures how the container is run. The most important parameter to set is `runOptions`, which controls which directories on the host filesystem are visible inside the container.

```
singularity {
  enabled    = true
  autoMounts = true
  cacheDir   = "${projectDir}/images"
  runOptions = "-B /path/to/your/data"
}
```

Set `runOptions` to bind-mount the parent directory that contains your data. For example, if your data lives under `/mnt/lab/projects/`, set:

```
runOptions = "-B /mnt/lab/projects"
```

If your working directory and data live in different top-level directories, add multiple bind mounts:

```
runOptions = "-B /mnt/isilon/hayeckteam -B /scr1/users/myname/data"
```

7. Defining Cell Line Comparisons (SamplePair.tsv)

MoDIFI computes pairwise comparisons between a **target** cell line and one or more **reference** cell lines. `SamplePair.tsv` defines which comparisons to run. There are three ways this file is handled:

Case I — You provide `SamplePair.tsv` before running

Create a tab-separated file with Target and Reference columns (Reference can be comma-separated for multiple references per target):

Target	Reference
GM12878	IMR90
IMR90	GM12878

If all required input data are found, the pipeline adds a PASS in the Check column and proceeds.

Case II — You provide SamplePair.tsv but some data is missing

The pipeline regenerates SamplePair.tsv with a Check column explicitly listing which data files are missing. Fix the missing files and re-run.

Case III — No SamplePair.tsv provided

If SamplePair.tsv is absent from resources/ when the pipeline starts, MoDIFI automatically generates all possible pairwise comparisons across every cell line named in RepNUM_atac and RepNUM_rna. This is the most computationally intensive option.

7.1 Recalculating MoDIFI for a new pair (without re-running DESeq2)

If you want to compute MoDIFI scores for a new target/reference pair using data that has already been preprocessed, you can skip the DESeq2 step:

7. Edit resources/SamplePair.tsv to add the new pair.
8. Re-run using the recalMoDIFI entry point:

```
nextflow run modifi.nf -c modifi_example.config -entry recalMoDIFI
```

8. Checking Intermediate Files (SampleInfo.tsv)

After the initial preprocessing and DESeq2 steps complete, the pipeline writes resources/SampleInfo.tsv. This file is the pipeline's internal record of where all intermediate files are. You do not need to edit it, but it is useful for diagnosing errors.

Specifically, SampleInfo.tsv tells you:

- **Whether SamplePair.tsv was correctly configured** — each row should show PASS in the Check column.
- **Whether all intermediate DESeq2 output files were found** — the Filename column lists the expected path for each omics comparison.

If any row shows something other than PASS, the pipeline will not proceed to the MoDIFI scoring step for that pair. Check:

- That the data files exist at the paths specified in modifi.config.
- That the Singularity bind mount (Section 6.3) covers the parent directory of those paths.

- That cell line names in the config match the directory names on disk (names are lowercased automatically).

9. Running the Pipeline

9.1 Full run (first time or after new data)

```
nextflow run modifi.nf -c modifi.config
```

9.2 Recalculate MoDIFI scores only (no DESeq2 rerun)

```
nextflow run modifi.nf -c modifi.config -entry recalMoDIFI
```

9.3 Resuming a failed or interrupted run

Add the `-resume` flag to restart from where the pipeline left off, reusing cached intermediate results:

```
nextflow run modifi.nf -c modifi.config -resume
```

10. Understanding the Output

All output files are written to the directory specified by `output_dir` in your config file.

10.1 Final MoDIFI output files

The primary output files are named `MoDIFI_all_[Target]_vs_[Reference].tsv`. Each row represents one Hi-C loop and its associated MoDIFI score for that cell line comparison.

The table summarizes each column in the MoDIFI output TSV, and the corresponding biological and analytical meanings are provided in *Column Dictionary_MoDIFI_data4567.docx* within *Supplemental_Data*.

10.2 Intermediate files

The pipeline also writes intermediate files that are useful for QC and debugging:

File pattern	Description
*_vs_*_ATACseq.txt	DESeq2 differential accessibility results for each cell line pair

File pattern	Description
*_vs_*_RNAseq.txt	DESeq2 differential expression results for each cell line pair for RNAseq
*_vs_*_ATACseq.txt	DESeq2 differential expression results for each cell line pair for ATACseq
PRO_ATAC_*.tsv	ATAC peaks mapped to promoter regions
ToBacon_*.tsv	Pre-bacon correction statistics (intermediate files)
ToBacon*bacon*.tsv	Post-bacon corrected statistics (Bayes factor inputs)

11. Troubleshooting

11.1 'No such file or directory' errors even when files exist

This is often a Singularity bind mount issue. The container cannot see host paths that have not been explicitly mounted.

Solution: Add the parent directory of the missing file to runOptions in your config:

```
runOptions = "-B /path/to/your/data"
```

If data lives in multiple locations:

```
runOptions = "-B /path/one -B /path/two"
```

11.2 SamplePair.tsv shows missing data

Check that:

- File glob patterns in ATACBAMFiles, ATACBEDFile, RNACountFile, HiCLoopsFile resolve to actual files.
- Cell line directory names on disk match the lowercased version of the config keys.
- The resources_dir path is correctly bound into the Singularity container.

11.3 Getting help

Please open an issue on the GitHub repository: <https://github.com/tris-10/MoDIFI/issues>. When reporting a bug, include the Nextflow log (.nextflow.log) and the contents of resources/SampleInfo.tsv.

12. Advanced: Using a Custom Prior

i BACKGROUND

By default, MoDIFI uses Gnocchi z-scores as prior weights. Higher Gnocchi z indicates stronger negative selection. The pipeline converts raw z-scores to percentile ranks (ascending) so that more constrained regions get higher prior weight. If you want to use a different constraint metric or prior, you can modify the `runCalBF_weighted` function in `scripts/calMoDIFY.py`. The relevant line is:

```
Gnocchi[PER_COL] = Gnocchi['z'].rank(method='min', ascending=True) /  
Gnocchi.shape[0]
```

Replace 'z' with your column name and adjust the ascending parameter based on whether higher values in your prior indicate more or less constraint.

13. Citation

If you use MoDIFI in your research, please cite our paper (*Multi-omics Differential Inference for Functional Interpretation (MoDIFI): A Statistical Framework to Prioritize Cell Lines for Neurodevelopmental Variants.*) along with other resources used in the pipeline:

- Chen, S., Francioli, L. C., Goodrich, J. K., Collins, R. L., Kanai, M., Wang, Q., Alföldi, J., Watts, N. A., Vittal, C., Gauthier, L. D., Poterba, T., Wilson, M. W., Tarasova, Y., Phu, W., Grant, R., Yohannes, M. T., Koenig, Z., Farjoun, Y., Banks, E., ... Karczewski, K. J. (2023). A genomic mutational constraint map using variation in 76,156 human genomes. *Nature*, (March 2022). <https://doi.org/10.1038/s41586-023-06045-0>
- Love, M. I., Huber, W., & Anders, S. (2014). Moderated estimation of fold change and dispersion for RNA-seq data with DESeq2. *Genome Biology*, 15(12), 1–21. <https://doi.org/10.1186/s13059-014-0550-8>
- van Iterson, M., van Zwet, E. W., & Heijmans, B. T. (2017). Controlling bias and inflation in epigenome- and transcriptome-wide association studies using the empirical null distribution. *Genome Biology*, 18(1), 19. <https://doi.org/10.1186/s13059-016-1131-9>
- VR, A., Shaw, G. T., Manuel, J., Mosbrugger, T. L., Heins, H., Ng, J. K., Kim, H., Hayeck, T. J., Turner, T. N., & Co-Senior Authors*. (2026). *Multi-omics Differential Inference for Functional Interpretation (MoDIFI): A Statistical Framework to Prioritize Cell Lines for Neurodevelopmental Variants.* <https://doi.org/10.64898/2026.01.29.702065>